МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине

«Облачные и туманные вычисления»

Выполнил:
Студент группы P3411
Болорболд Аригуун

Преподаватель:
Перл Ольга Вячеславовна

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Ход работы 4
   2.1. Подготовка окружения 4
   2.2. Создание манифестов `Kubernetes` 4
   2.3. Redis 5
   2.4. Python 6
   2.5. Развертывание 7
   2.6. Проверка работы 8
   2.7. Rolling Update 9
3. Вывод 11

1. Текст задания

Цель:

Перенести нагрузку из Лабораторной №1 в `Kubernetes`. Так как мы избегаем `OpenShift`, используйте `Minikube` или `Kind` (`Kubernetes in Docker`). Это научит работать с примитивами (`Pods`, `Deployments`, `Services`), которые `OpenShift` обычно скрывает за абстракциями.

Инструменты: `Minikube` (или Kind), `kubectl`.

Ход работы:

1. **Трансляция (Translation)**: Вы должны вручную «перевести» логику `Docker Compose` в манифесты `K8s`:
2. **Развертывание**: Применить манифесты к локальному кластеру (`kubectl apply`).
3. **Rolling Update (Обновление без простоя)**: Вы меняете код `Python` (например, цвет фона), собирают образ версии v2, обновляют YAML-файл и наблюдают, как `Kubernetes` выполняет постепенное обновление подов без остановки сервиса.

Ключевые знания: Жизненный цикл `Pod`'а, разница между императивным и декларативным подходом, внутренняя сеть кластера.

2. Ход работы

2.1. Подготовка окружения

Создание кластера показано на рисунке 1:

```
kind create cluster --name lab2
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> kind create cluster --name lab2
Creating cluster "lab2" ...
 • Ensuring node image (kindest/node:v1.35.0) 🖼 ...
 ✓ Ensuring node image (kindest/node:v1.35.0) 🖼
 • Preparing nodes 📦  ...
 ✓ Preparing nodes 📦
 • Writing configuration 📜 ...
 ✓ Writing configuration 📜
 • Starting control-plane 🕹️ ...
 ✓ Starting control-plane 🕹️
 • Installing CNI 🔌 ...
 ✓ Installing CNI 🔌
 • Installing StorageClass 💾 ...
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab2"
You can now use your cluster with:

kubectl cluster-info --context kind-lab2

Not sure what to do next? 😅 Check out https://kind.sigs.k8s.io/docs/user/quick-start/
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2>
```

Рисунок 1 — Создание кластера `kind`

2.2. Создание манифестов `Kubernetes`

Конфигурация (`k8s/config.yaml`):

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
type: Opaque
stringData:
  REDIS_PASSWORD: password
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  REDIS_HOST: redis
  REDIS_PORT: "6379"
  REDIS_DB: "0"
```

2.3. Redis

Deployment для Redis (k8s/redis-deployment.yaml):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        envFrom:
        - secretRef:
            name: app-secrets
        command:
        - sh
        - -c
        - redis-server --appendonly yes --requirepass
$REDIS_PASSWORD
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
      volumes:
      - name: redis-data
        emptyDir: {}
```

Service для Redis (k8s/redis-service.yaml):

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  type: ClusterIP
```

```yaml
  selector:
    app: redis
  ports:
  - port: 6379
    targetPort: 6379
```

2.4. Python

`Deployment` для приложения (`k8s/app-deployment.yaml`):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-app
  labels:
    app: python-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: python-app
  template:
    metadata:
      labels:
        app: python-app
    spec:
      containers:
      - name: python-app
        image: lab2-app:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
        envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: app-secrets
```

`Service` для приложения (`k8s/app-service.yaml`):

```yaml
apiVersion: v1
kind: Service
metadata:
  name: python-app-service
spec:
  type: NodePort
  selector:
    app: python-app
```

```yaml
  ports:
  - port: 5000
    targetPort: 5000
    nodePort: 31010
```

2.5. Развертывание

Сборка `Docker`-образа приложения и загрузка его в кластер `Kind`:

```bash
docker build -t lab2-app:v1 .
kind load docker-image lab2-app:v1 --name lab2
```

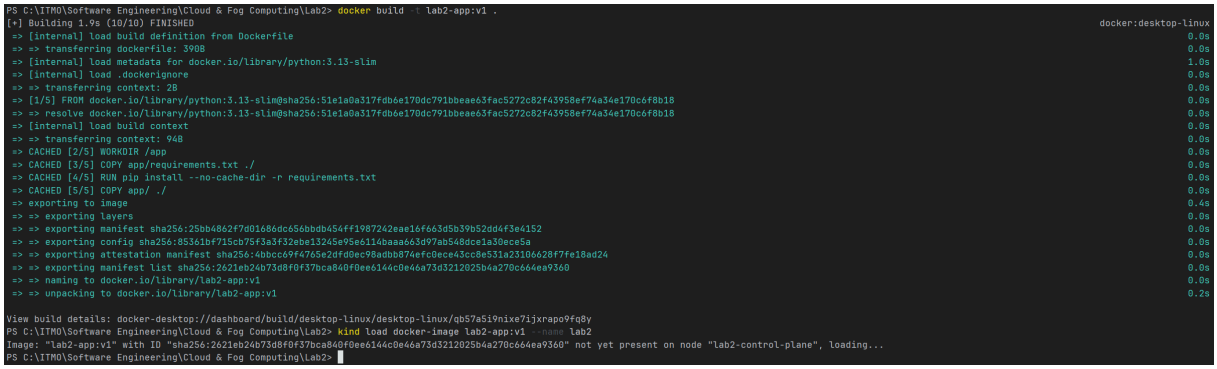

Рисунок 2 — Сборка `Docker`-образа приложения и загрузка его в кластер `Kind`

Применение манифестов:

```bash
kubectl apply -f k8s/config.yaml
kubectl apply -f k8s/redis-deployment.yaml
kubectl apply -f k8s/redis-service.yaml
kubectl apply -f k8s/app-deployment.yaml
kubectl apply -f k8s/app-service.yaml
```

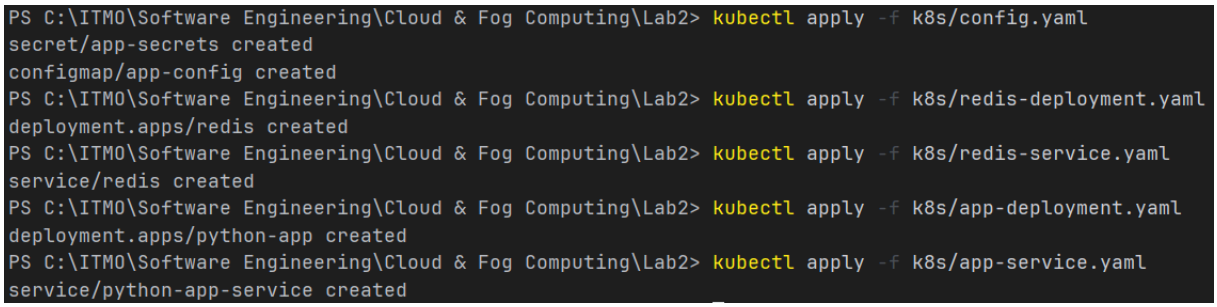

Рисунок 3 — Применение манифестов

Проверка статуса подов:

```bash
kubectl get pods
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
python-app-74ddf554db-7k685   1/1     Running   0          68s
python-app-74ddf554db-htl2s   1/1     Running   0          68s
python-app-74ddf554db-mcxp4   1/1     Running   0          68s
redis-8458556757-5xmgz        1/1     Running   0          81s
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2>
```

Рисунок 4 — отображение статуса подов

2.6. Проверка работы

Для доступа к приложению с хост-машины использовался проброс портов, так как `Kind` запускает узлов `Docker`-контейнерах

```
kubectl port-forward service/python-app-service 9000:5000
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> kubectl port-forward service/python-app-service 9000:5000
Forwarding from 127.0.0.1:9000 -> 5000
Forwarding from [::1]:9000 -> 5000
Handling connection for 9000
Handling connection for 9000
Handling connection for 9000
```

Рисунок 5 — Проброс порта для проверки доступности

```
curl.exe http://localhost:9000/
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe -s http://localhost:9000/api
{"version":"v1","visit_count":1}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe -s http://localhost:9000/api
{"version":"v1","visit_count":1}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe -s http://localhost:9000/api
{"version":"v1","visit_count":1}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe -s http://localhost:9000/api
{"version":"v1","visit_count":1}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2>
```

Рисунок 6 — Проверка доступности через `curl`

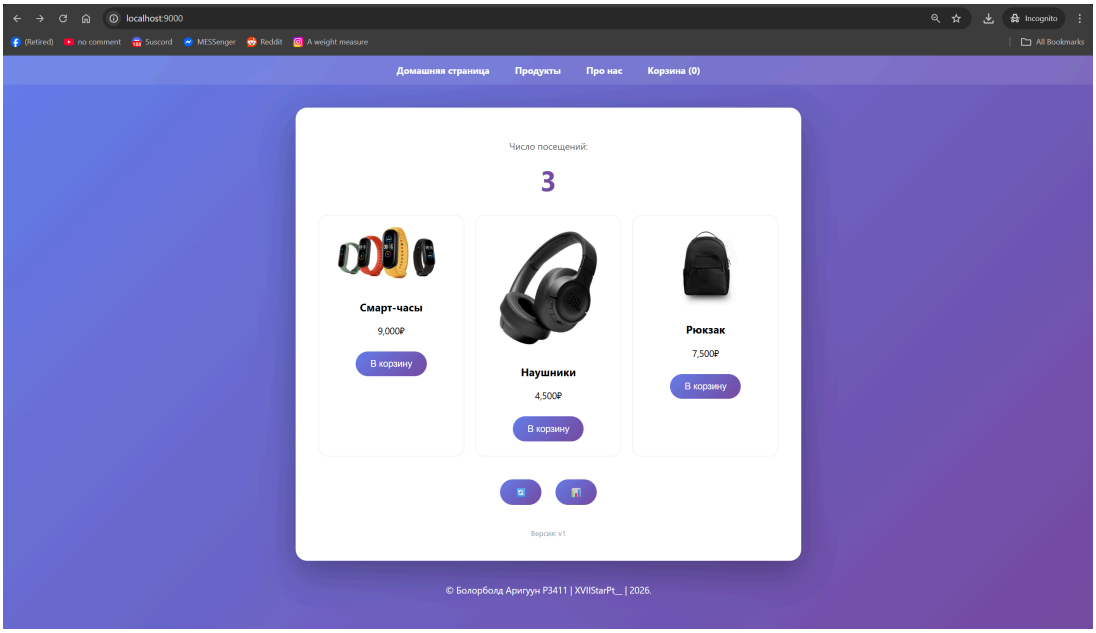

Рисунок 7 — Проверка доступности через браузер

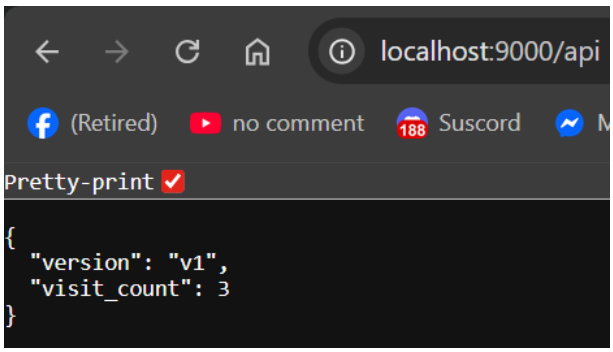

Рисунок 8 — Проверка доступности через браузер (/api)

2.7. Rolling Update

Для демонстрации обновления без простоя был изменен код приложения (внешний вид веб-сайта):

```
background: linear-gradient(135deg, #11998e 0%, #38ef7d 100%);
color: #11998e;
color: #38ef7d;
background: linear-gradient(135deg, #11998e 0%, #38ef7d 100%);
```

```
version = os.getenv("APP_VERSION", "v2")
```

Сборка новой версии образа показана на рисунке 8:

```
docker build -t lab2-app:v2 .
kind load docker-image lab2-app:v2 --name lab2
```

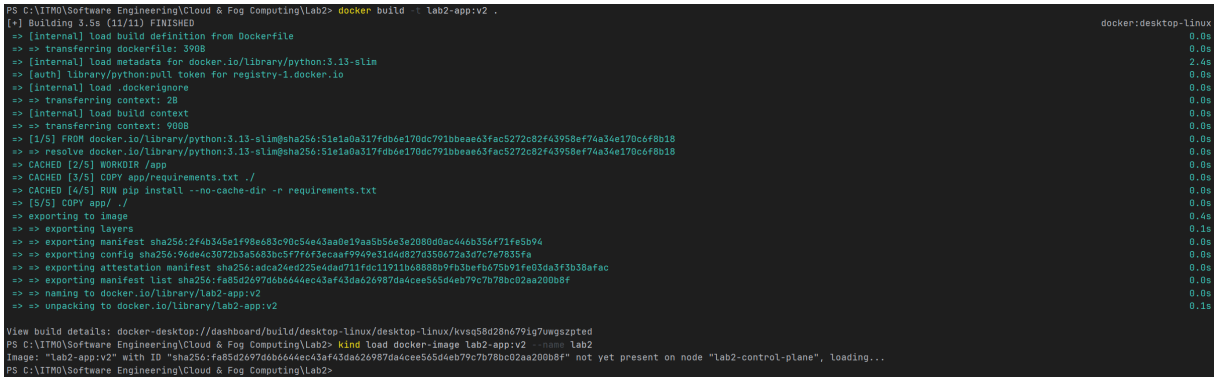

Рисунок 8 — Сборка новой версии образа

После чего следовало изменение версии образа в манифесте и повторное применение конфигурации.

Теперь если наблюдать за процессом обновления:

```
kubectl rollout status deployment/python-app
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> kubectl rollout status deployment/python-app
Waiting for deployment "python-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "python-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "python-app" rollout to finish: 1 old replicas are pending termination...
deployment "python-app" successfully rolled out
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2>
```

Рисунок 9 — Наблюдение за процессо обновления

Проверка новой версии:

```
curl.exe http://localhost:9000/
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe http://localhost:9000/api
{"version":"v2","visit_count":6}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe http://localhost:9000/api
{"version":"v2","visit_count":6}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe http://localhost:9000/api
{"version":"v2","visit_count":6}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2> curl.exe http://localhost:9000/api
{"version":"v2","visit_count":6}
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab2>
```

Рисунок 10 — Проверка доступности новой версии через `curl`

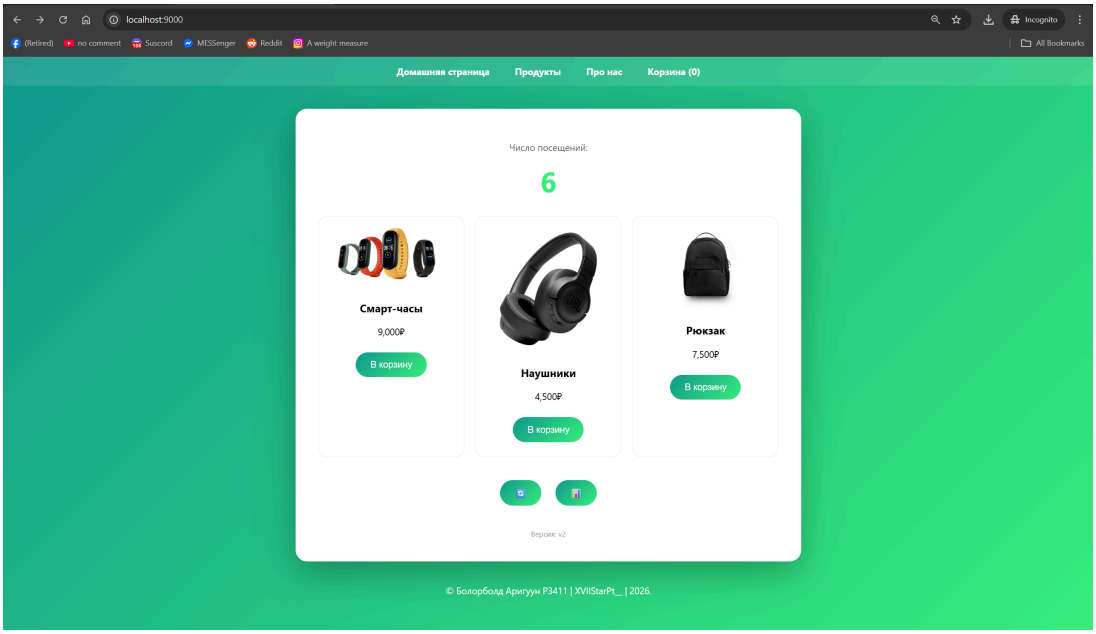

Рисунок 11 — Проверка доступности новой версии через браузер

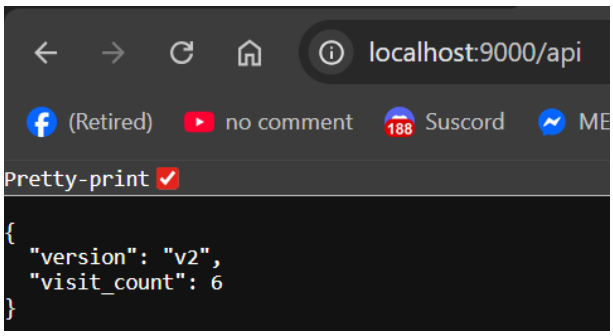

Рисунок 12 — Проверка доступности новой версии через браузер (`/api`)

Приложение успешно обновилось, счетчик посещений сохранился (т.к. `Redis` вынесен в отдельный сервис), и сервис оставался доступным во время обновления.

3. Вывод

В ходе лабораторной работы веб-приложение было перенесено из изолированной среды `Docker Compose` в оркестрируемый кластер `Kubernetes`. Ключевые освоенные концепции и примитивы `Kubernetes`:

- `Pods` **(поды)**: Базовые единицы развертывания.
- `Deployments` **(деплойменты)**: Управление жизненным циклом подов и их целевым состоянием.
- `Services` **(сервисы)**: Обеспечение сетевого доступа к приложению.

Особый акцент был сделан на успешном тестировании стратегии **Rolling Update (постепенное обновление)**. Эта стратегия позволила обновить версию приложения без прерывания его доступности. Был продемонстрирован декларативный подход с использованием YAML-манифестов, который показал явное преимущество в управлении инфраструктурой по сравнению с императивным запуском контейнеров.